Мультиплексирование ввода-вывода

Илья Кокорин

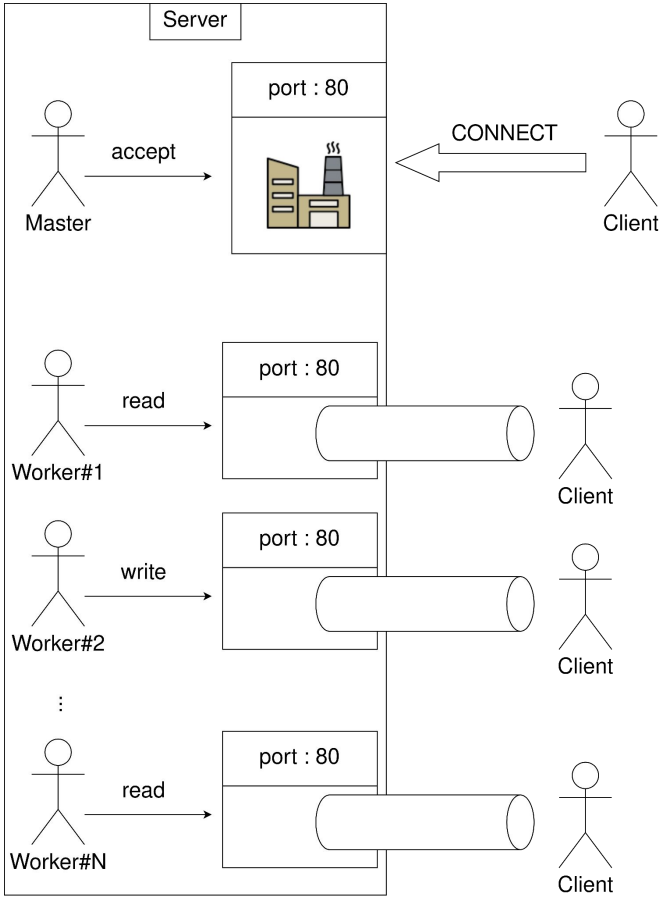
kokorin.ilya.1998@gmail.com

TCP: многопоточный сервер

- Обрабатываем каждого клиента в отдельном потоке

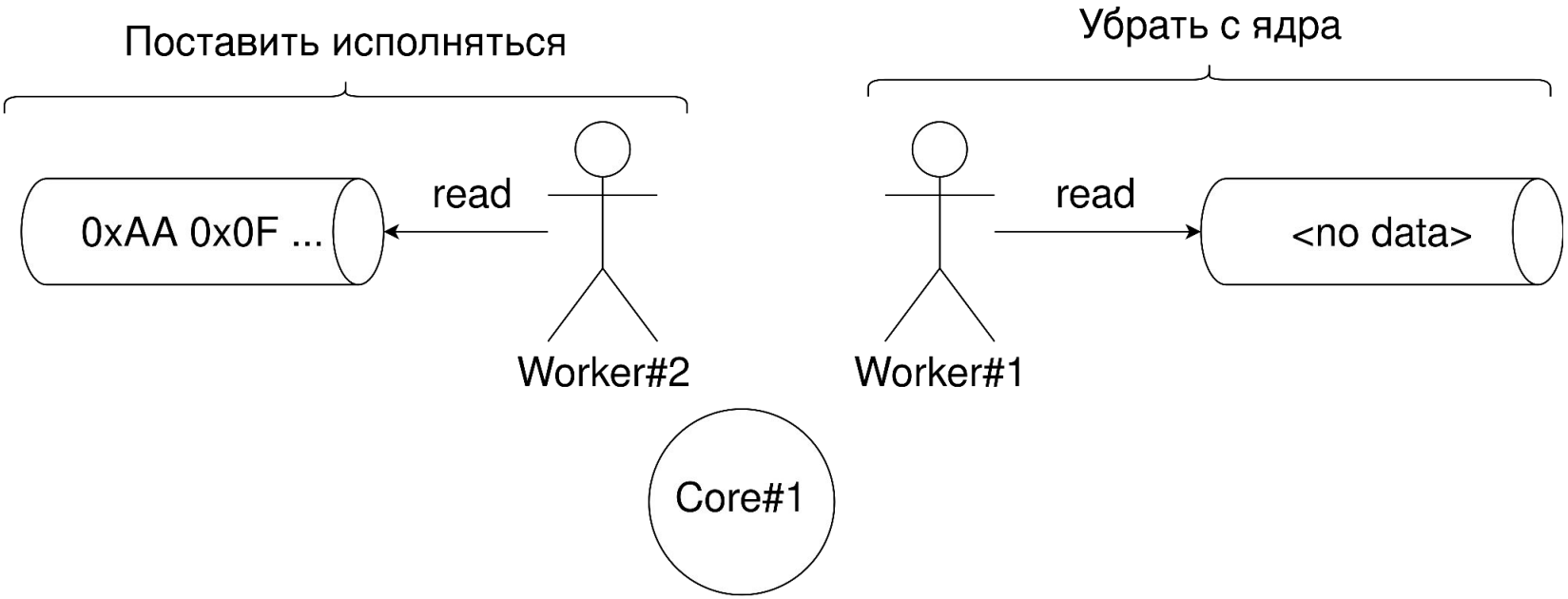
```
fd = socket(IPPROTO_TCP)
bind(fd, host, port)
listen(fd, bqs)
while true:
    conn := accept(fd)
    thread {
        process_client(conn)
        close(conn)
    }
close(fd)
```

TCP: многопоточный сервер



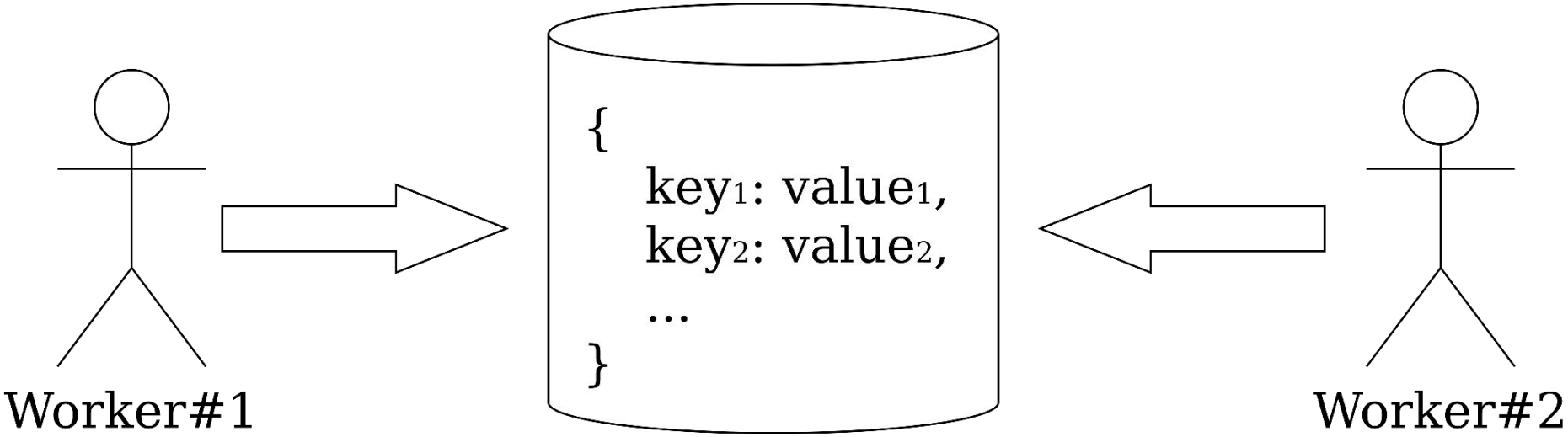
Цена потока

- Поток занимает оперативную память
- context switch не бесплатные



Цена потока

- Сложность обновления общего состояния
- Вы знаете это с параллельного программирования

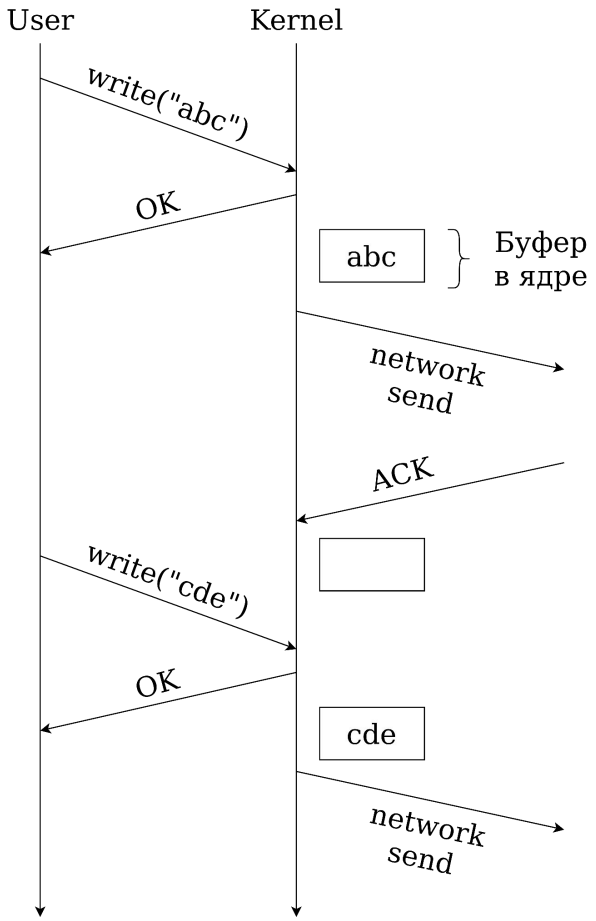


Неблокирубщие сокеты

- `fcntl(fd, O_NONBLOCK)`
- `data, ok := try_read(fd, size)`
  - `ok = false` если данных пока нет
- `ok := try_write(fd, buf, offset, size)`
  - `ok = false` если записать не получается
  - Если сетевой стек ОС занят отправкой других данных
- `conn, ok := try_accept(fd)`
  - `ok = false` если новый клиент пока не пришёл
- И т.д.

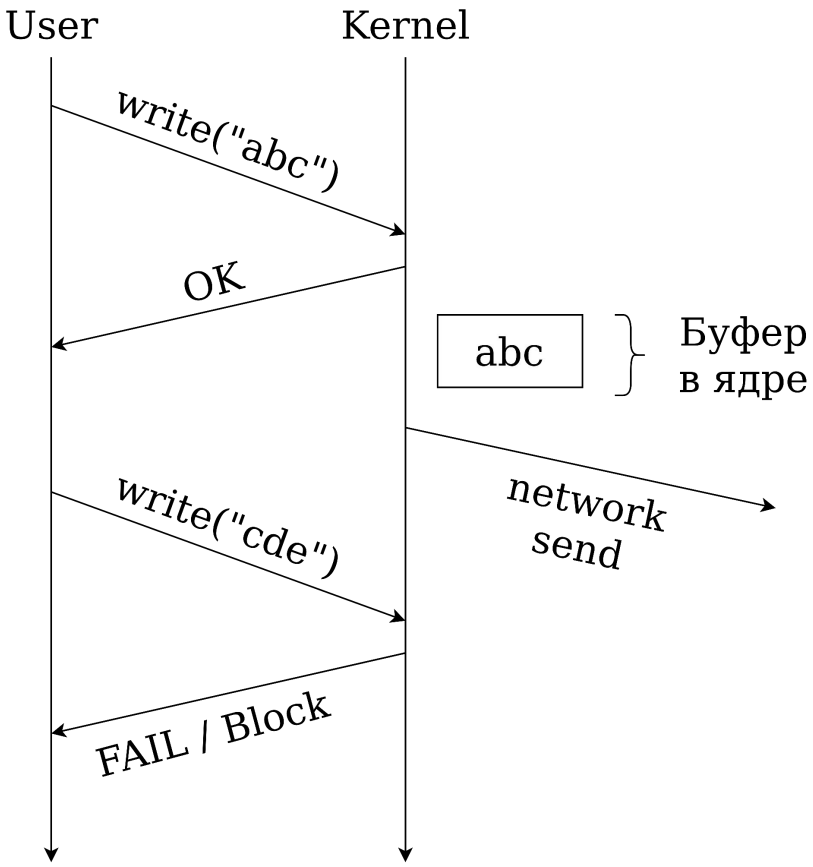
Запись в TCP сокет

- У TCP сокета под капотом есть буфер
- В него записывается очередной кусок данных
- ОС отправляет его
- И ждёт подтверждения получения
- Перепосылает, если нужно
- Освобождает буфер при получении подтверждения
- В него можно записывать новые данные



Запись в TCP сокет

- В буфере может не быть места
- Потому что не пришло подтверждение приёма предыдущей порции данных
- Новую порцию данных записать в буфер нельзя
- Синхронный сокет заблокируется
- Асинхронный — вернёт ошибку



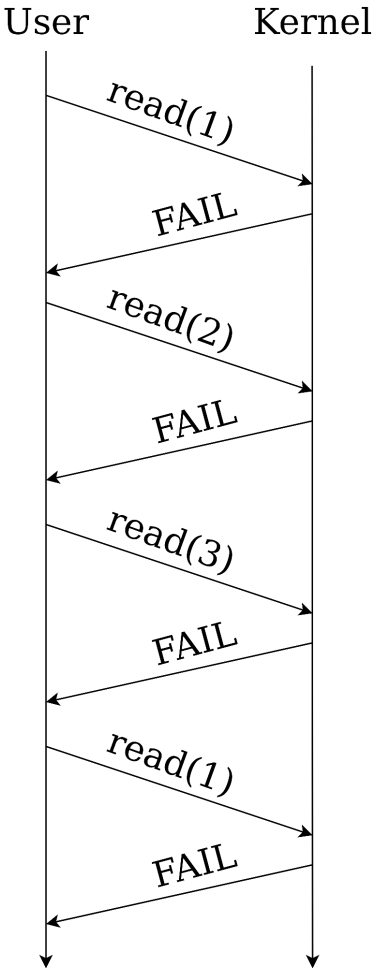
Неблокирубщие сокеты

- Пробуем работать с каждым сокетом по кругу
- Переходим к следующему, если не получилось

```
fd := socket(IPPROTO_TCP | O_NONBLOCK)
bind(fd, host, port)
listen(fd, bqs)
state := initial_state()
conns := []
while true:
    conn, ok := try_accept(fd, O_NONBLOCK)
    if ok:
        conns.append(conn)
    for conn in conns:
        data, ok := try_read(conn)
        if ok:
            state ← change_state(state, data)
    ...
```

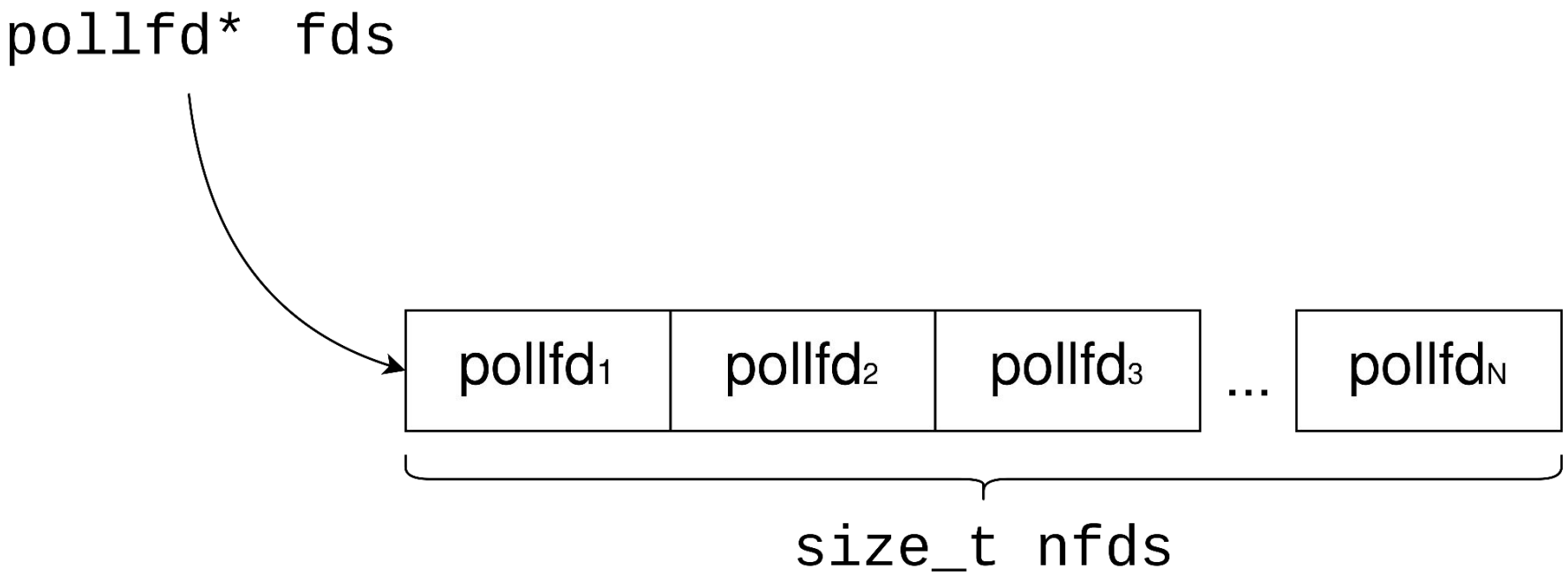
Неблокирубщие сокеты

- Активное ожидание
- Поллинг ОС
- Впустую тратим CPU
- Хочется чтобы ОС сама оповещала нас, когда в интересующем нас сокете появятся данные
  - Или возможность их записать
  - Или новый клиент
  - И т.д.



poll

- Системный вызов
- `int poll(pollfd* fds, size_t nfds, int time);`



poll

- Структура `pollfd` состоит из трёх полей

```
/* Data structure describing a polling request.  */
struct pollfd
  {
    int fd;                     /* File descriptor to poll.  */
    short int events;           /* Types of events poller cares about.  */
    short int revents;          /* Types of events that actually occurred.  */
  };
```

- `fd` это номер файлового дескриптора, на котором мы хотим ждать возможности выполнения операции
  - Чтения/записи/нового клиента
- Два оставшихся поля это битовые маски

poll: битовые маски

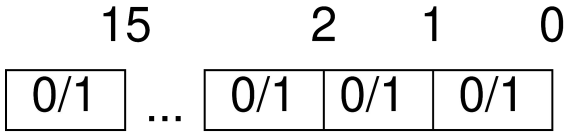
- `short` состоит из 16 бит
- Каждый бит отвечает за какую-то операцию
  - 0 — за чтение и приём новых клиентов
  - 2 — за запись
  - 3 — за ошибки
  - 4 — за закрытие соединения
  - И т.д.

15		2	1	0
0/1	...	0/1	0/1	0/1

- Мы можем хотеть получить какое-то множество событий
  - Говорим об этом, ставя биты в `events`
  - `pollfd.events = (1 << 0) | (1 << 2)`
  - `pollfd.events = POLLIN | POLLOUT`

poll: битовые маски

- `short` состоит из 16 бит
- Каждый бит отвечает за какую-то операцию
  - 0 — за чтение и приём новых клиентов
  - 2 — за запись
  - 3 — за ошибки
  - 4 — за закрытие соединения
  - И т.д.



- Аналогичным образом в `revents` ядро возвращает, какие события произошли с дескриптором
  - `(pollfd.revents & (1 << 2)) > 0`
  - `(pollfd.revents & POLLOUT) > 0`

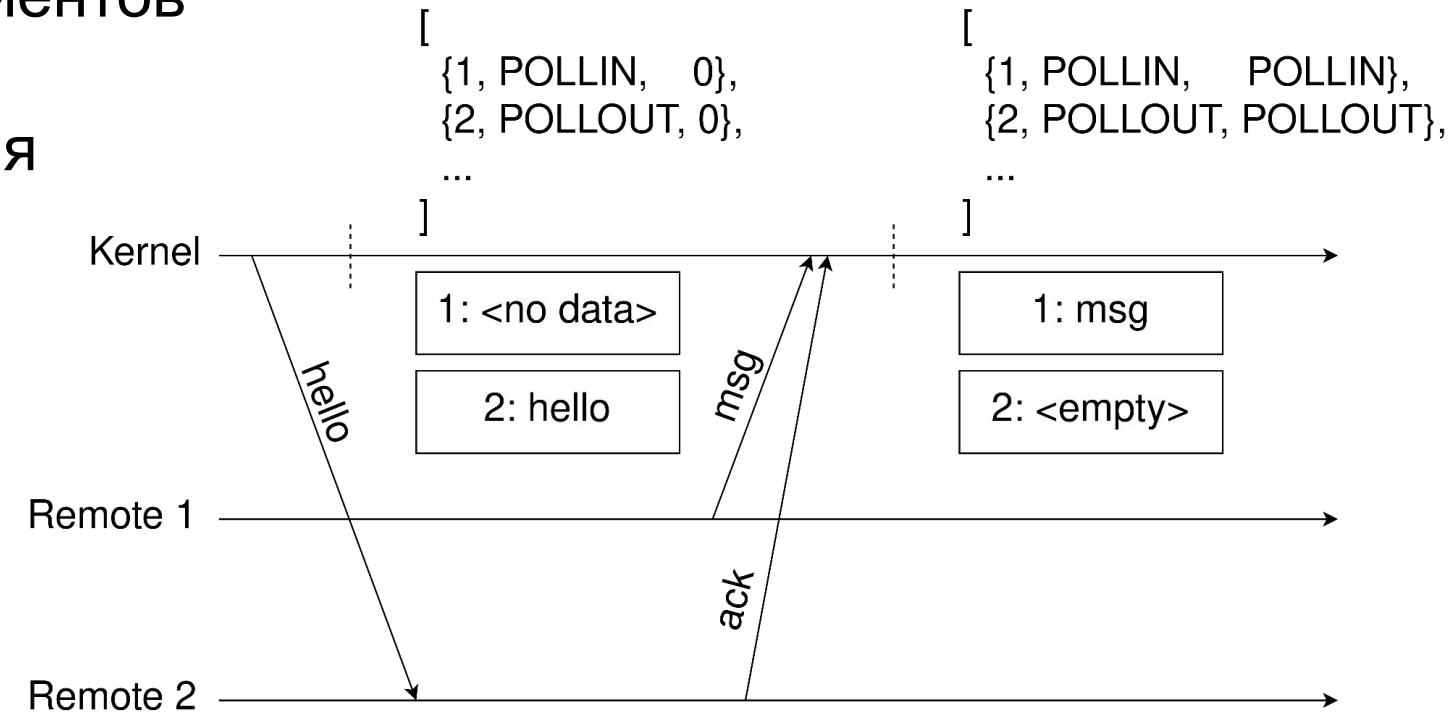
poll

```
fds = []
for conn in conns:
    cur_flag = 0
    if should_read:
        cur_flag |= POLLIN
    if should_write:
        cur_flag |= POLLOUT
    if cur_flag == 0:
        continue
    fds.append({ fd = cur_fd, events = cur_flag, revents = 0 })

poll(fds)
for cur_fd in fds:
    if cur_fd.revents & POLLIN != 0:
        process_read(cur_fd.fd)
    if cur_fd.revents & POLLOUT != 0:
        process_write(cur_fd.fd)
```

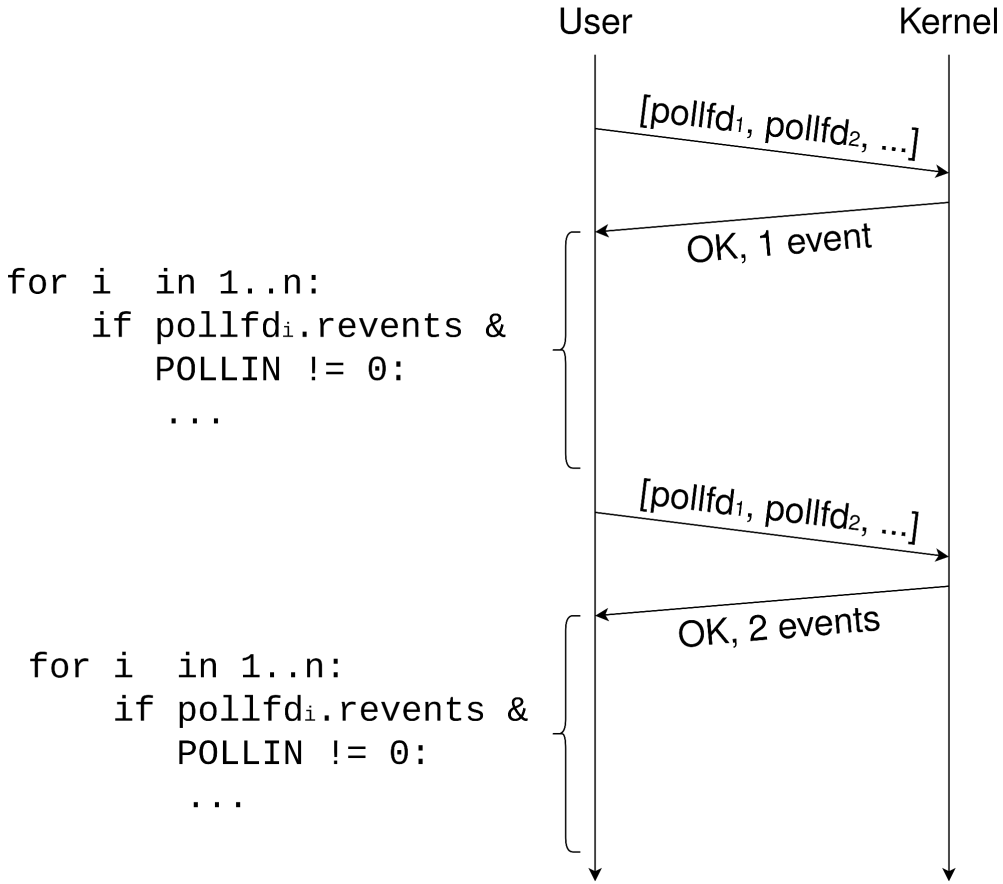
poll: внутреннее устройство

- Реагирует на освобождение буферов для записи
- Получение данных
- Новых клиентов
- И прочие сообщения из сети



poll: проблемы с масштабируемостью

- Очень много времени тратится на обход элементов массива с `revents = 0`



epoll: решение проблемы

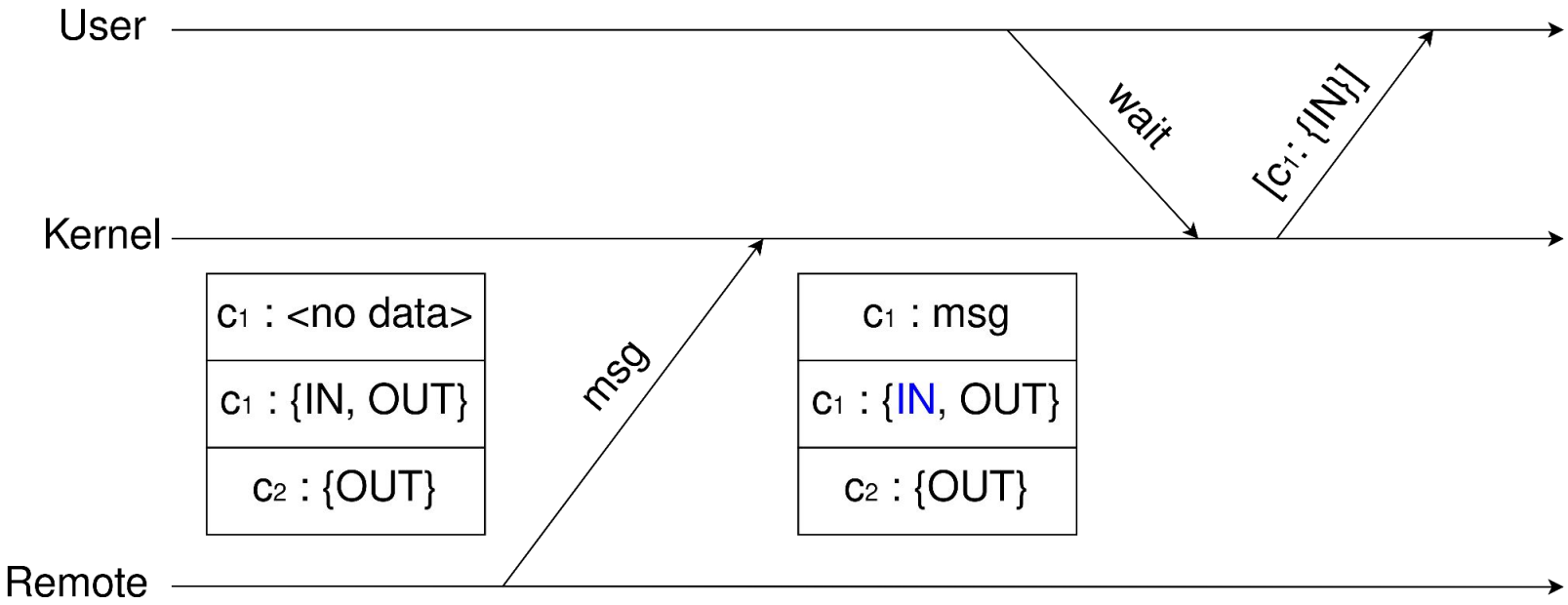
- Будем хранить множество файловых дескрипторов в ядре
  - Для каждого дескриптора будем хранить ещё и множество интересующих событий
  - В виде битовой маски (`EPOLLIN | EPOLLOUT`)

epoll: операции

- `epoll := epoll_create()` — создать множество
- `epoll_ctl_add(epoll, fd, mask)` — добавить файловый дескриптор с маской событий
- `epoll_ctl_remove(epoll, fd)` — убрать файловый дескриптор из множества
- `epoll_ctl_set(epoll, fd, mask)` — изменить маску событий для файлового дескриптора
- `events := epoll_wait(epoll, timeout)` — ждать наступления событий

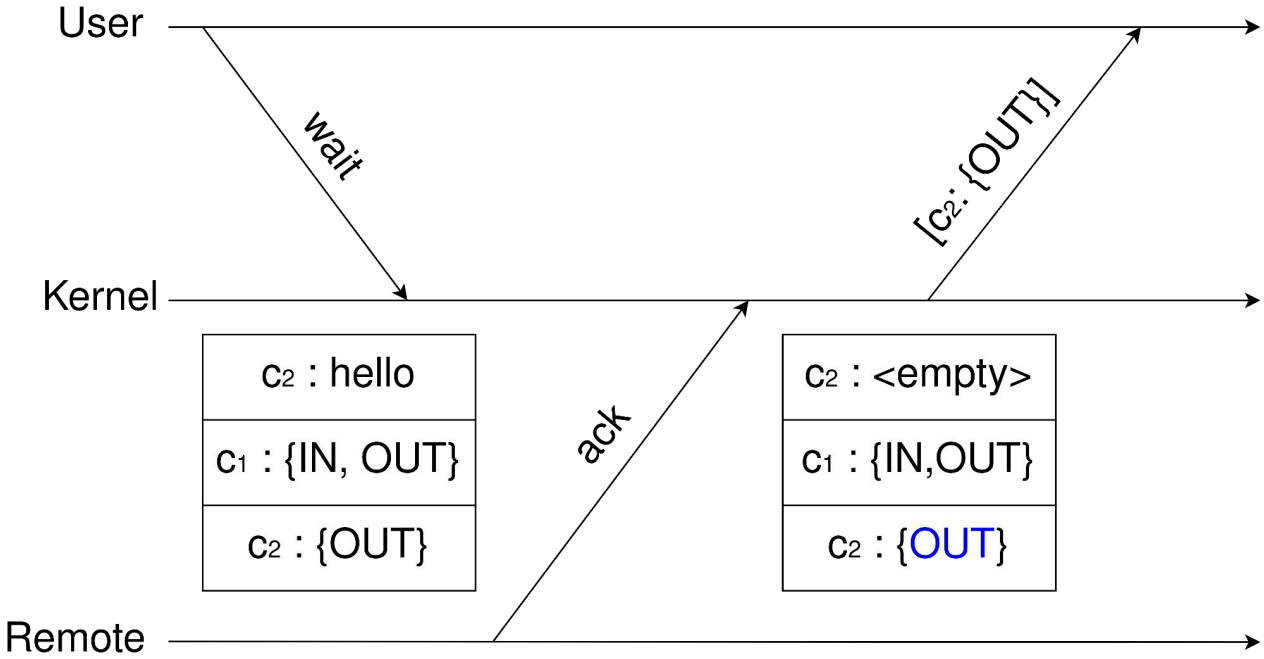
epoll

- Ядро само ждёт наступления интересных нам событий
- Помечает событие как произошедшее
- Сигнализирует пользователю, когда нужно



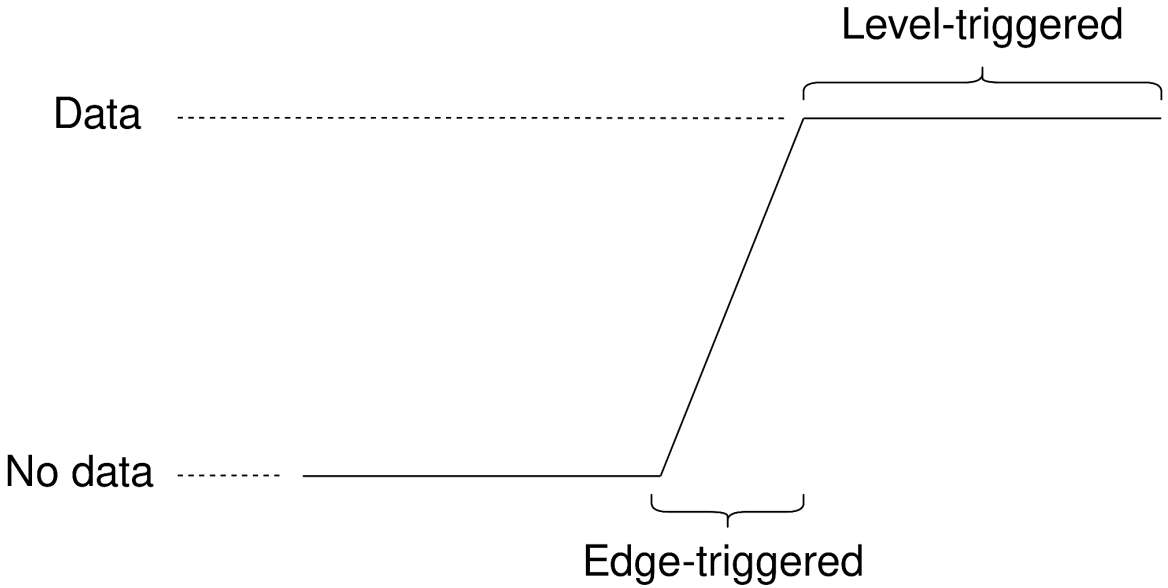
epoll

- Ядро само ждёт наступления интересных нам событий
- Помечает событие как произошедшее
- Сигнализирует пользователю, когда нужно



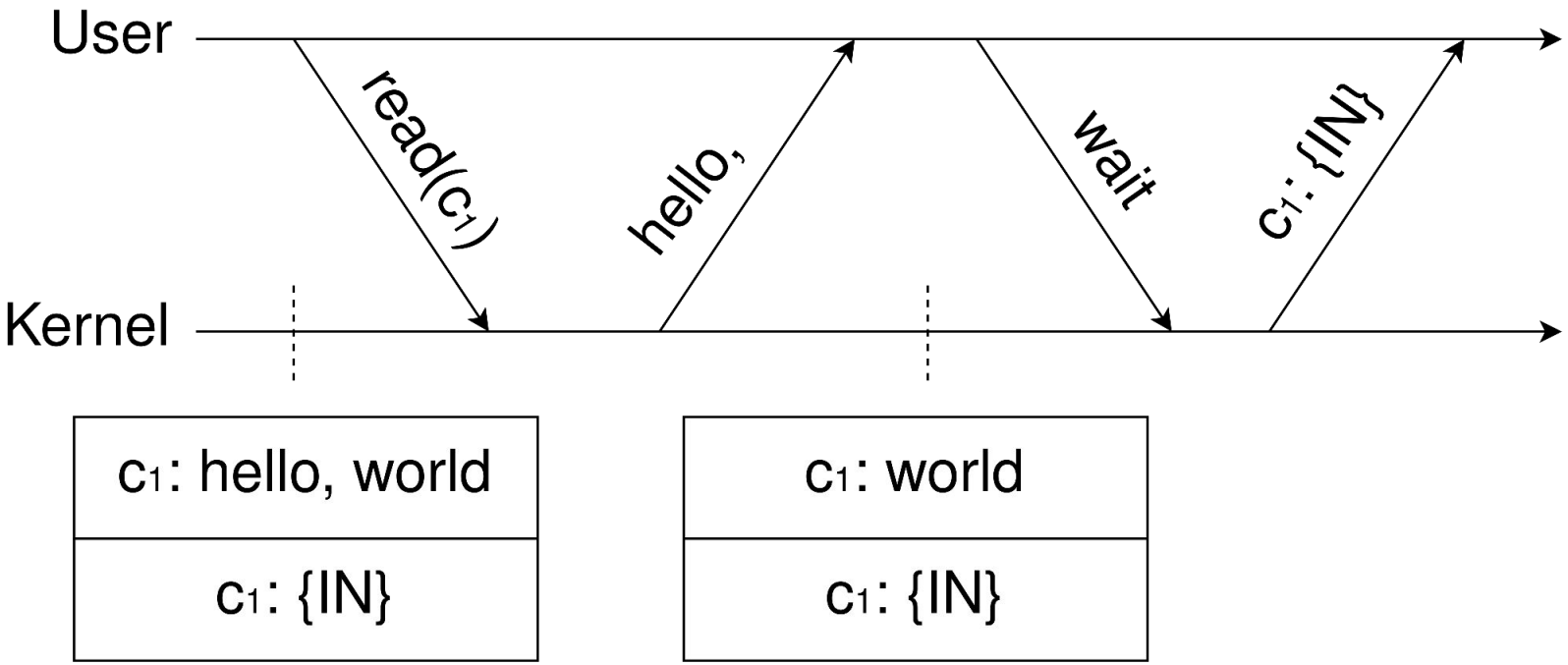
epoll: edge- vs level-triggered

- epoll работает в одном из двух режимов
- Level-triggered epoll отмечает готовность дескриптора если в буфере есть данные/место для записи
- Edge-triggered epoll отмечает, что дескриптор готов к совершению операции только в момент **появления новых** данных



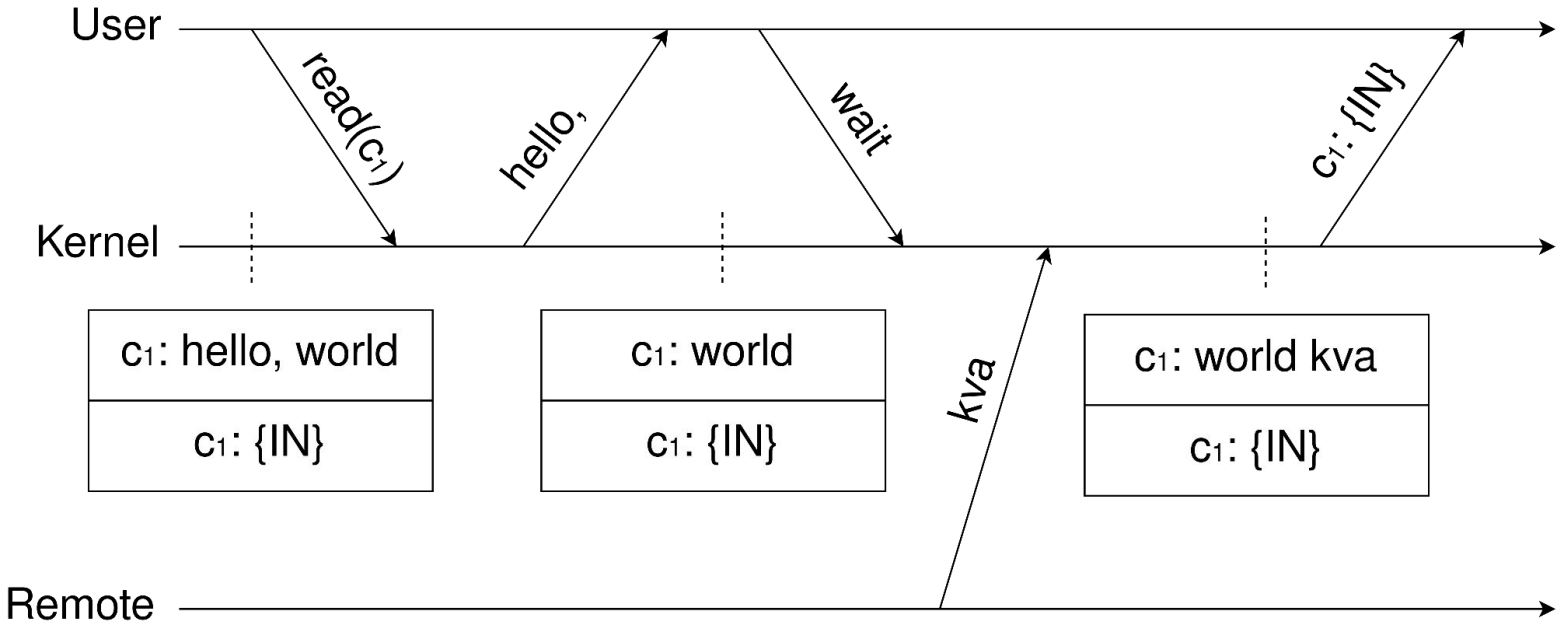
Level-triggered epoll

- Прочитали часть данных из буфера сокета
- Вызвали `epoll_wait`
- Ядро вернёт сокет как готовый к чтению



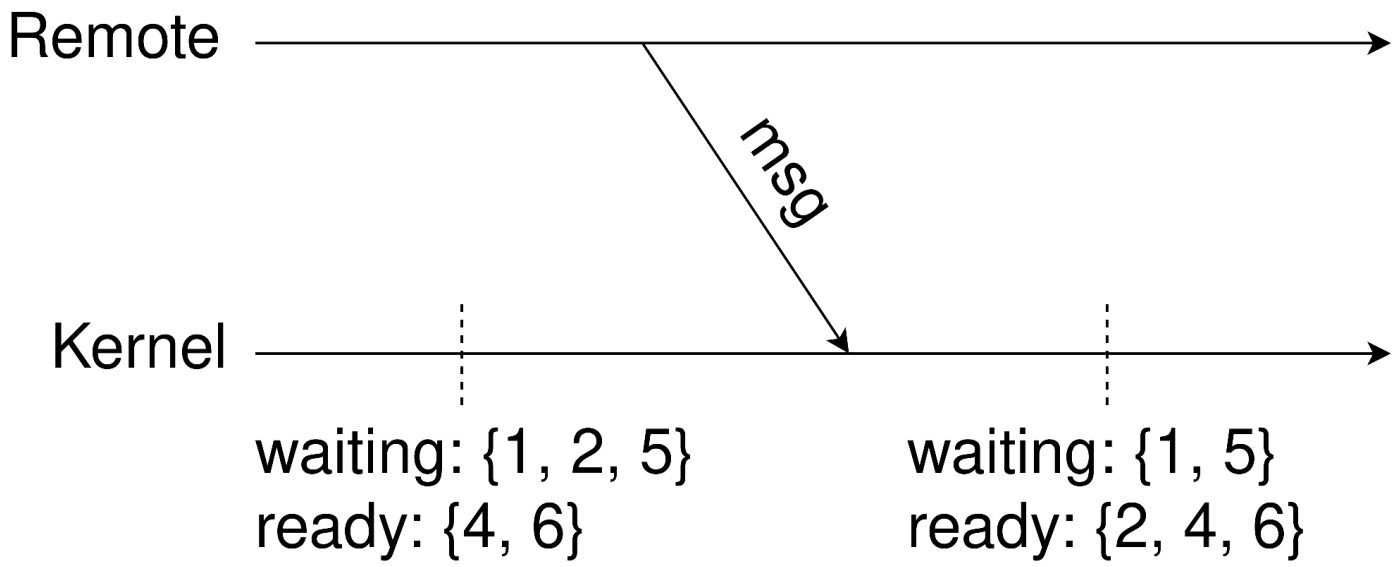
Edge-triggered epoll

- Ядро вернёт сокет как готовый к чтению пока в него не придут ещё данные
- Новые данные могут не прийти никогда



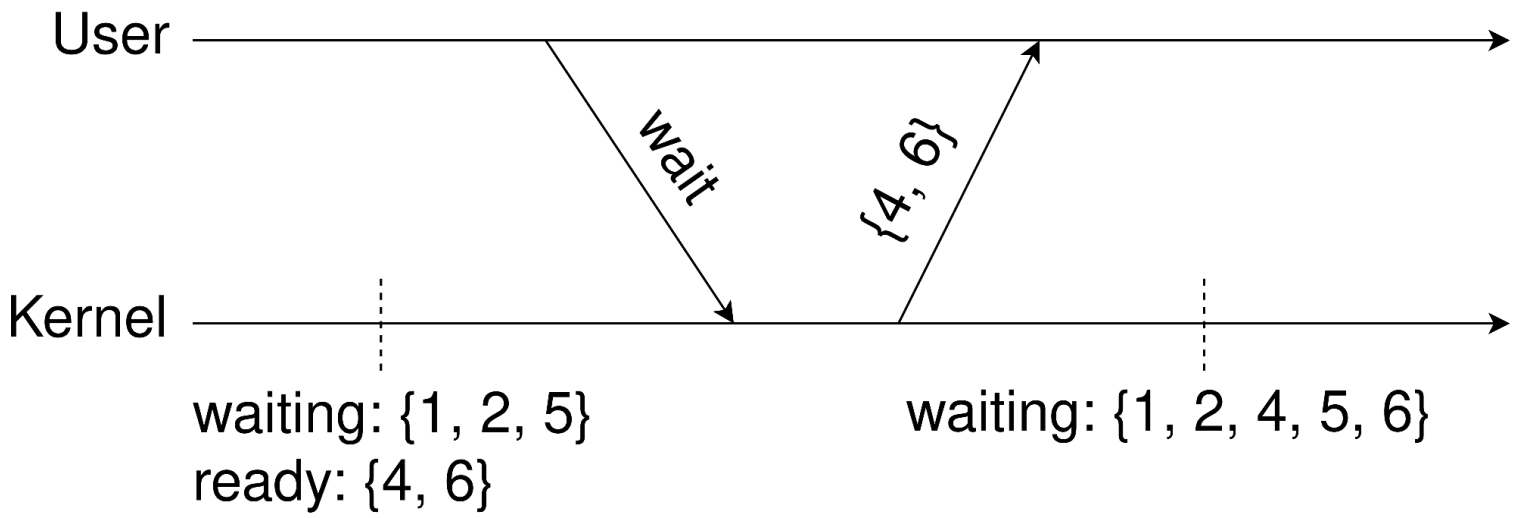
Edge-triggered epoll

- Может быть реализован очень эффективно
- Храним два множества: `ready` и `waiting`
- Из `waiting` в `ready` дескриптор перемещаем при получении сообщения



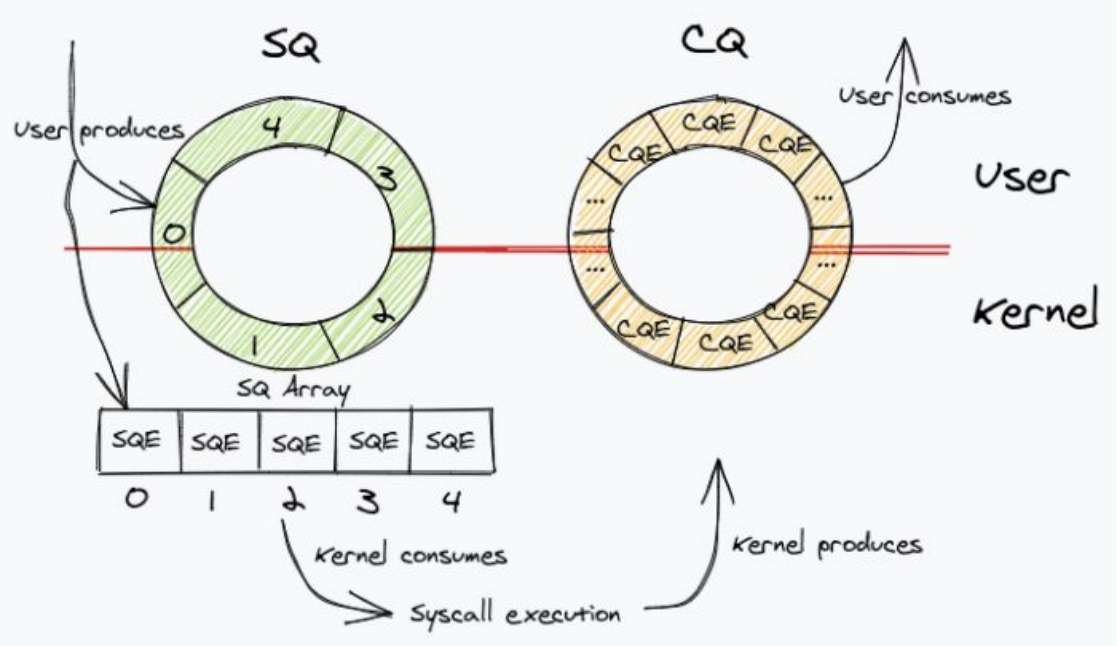
Edge-triggered epoll

- При epoll_wait от пользователя возвращаем ему все дескрипторы из ready
- Перемещаем все дескрипторы из ready в waiting
- В level-triggered режиме нужно перемещать только те, в которых не осталось данных



io_uring

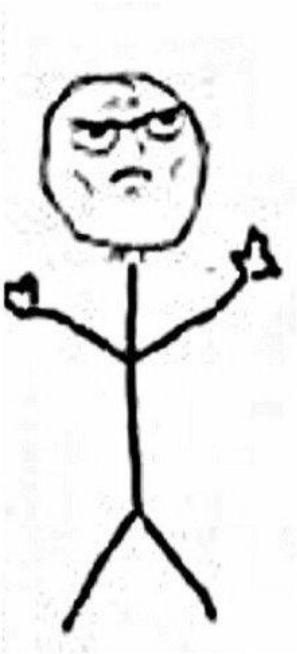
- Новое API мультиплексирования ввода-вывода в современных Linux
- Меньше копирования между ядром и пользователем
- И другие улучшения



Что почитать: вездесущий epoll

- epoll в boost::asio,
  - И ещё
- epoll в go runtime
- epoll в Java Netty,
  - И ещё
- epoll в Python Twister

Что почитать: мультиплексирование в целом

- Миллион вебсокетов и Go
- *Roman Elizarov.* introduction to coroutines in Kotlin
- The implementation of epoll
- *Eli Bendersky.* Concurrent Servers
- *Dan Kegel.* The C10K problem
- Вся правда о Linux epoll
- Why does one NGINX worker takes all the load
- Введение в io_uring
- Efficient IO with io_uring

спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание